

Projet Pratique — Éléments Finis

Résolution du problème de gauchissement de Saint-Venant
par éléments triangulaires CST en déformation plane

Oumkalthoum M'HAMDI

Année universitaire 2024–2025

Contents

1	Introduction	2
2	Formulation du problème	2
2.1	Problème physique — Torsion de Saint-Venant	2
2.2	Déformations et contraintes de cisaillement	3
2.3	Équation forte	3
2.4	Formulation variationnelle	4
3	Discrétisation par éléments finis	4
3.1	Approximation par éléments triangulaires CST	4
3.2	Gradient discret et matrice B	5
3.3	Formulation élémentaire	5
3.4	Vecteur de charge élémentaire	6
3.5	Assemblage global	6
3.6	Condition d'unicité	7
4	Implémentation Fortran	7
4.1	Structure du programme	7
4.2	Boucle d'assemblage	7
5	Résultats et interprétation	8
5.1	Maillage	8
5.2	Fonction de gauchissement ω	8
5.3	Profils de ω	9
5.4	Contraintes de cisaillement	9
6	Conclusion	10
A	Code source — TP_elas.f90	10

1 Introduction

Ce rapport présente la mise en œuvre de la méthode des éléments finis (MEF) pour la résolution du problème de gauchissement de Saint-Venant sur une section droite soumise à de la torsion. Le problème se ramène à la résolution de l'équation de Laplace sur la section transversale Ω , avec des conditions aux limites de Neumann sur le contour $\partial\Omega$.

Les objectifs du travail pratique sont :

- Formuler le problème variationnel associé à l'équation de Laplace,
- Discrétiser le domaine par des éléments triangulaires linéaires (CST),
- Implémenter l'assemblage de la matrice de rigidité et du vecteur de charge en Fortran 90,
- Résoudre le système linéaire et post-traiter les résultats.

2 Formulation du problème

2.1 Problème physique — Torsion de Saint-Venant

On considère une poutre prismatique droite d'axe longitudinal x , soumise à un moment de torsion M_t constant. La section transversale $\Omega \subset \mathbb{R}^2$ est repérée dans le plan (y, z) .

Dans le cadre de la théorie de Saint-Venant, on suppose :

- des petites déformations,
- un matériau élastique linéaire isotrope,
- une torsion uniforme le long de l'axe x ,
- des sections initialement planes pouvant toutefois se gauchir.

Contrairement à la torsion purement rigide, les sections ne restent pas strictement planes : elles subissent un déplacement axial supplémentaire appelé **gauchissement**.

Le champ de déplacement associé à une torsion uniforme s'écrit :

$$\begin{cases} u_x = \alpha \omega(y, z), \\ u_y = -\alpha x z, \\ u_z = \alpha x y, \end{cases} \quad (1)$$

où :

- α est le taux de torsion (rotation par unité de longueur),
- $\omega(y, z)$ est la fonction de gauchissement,
- (u_x, u_y, u_z) désignent les composantes du déplacement.

Les composantes u_y et u_z correspondent à une rotation rigide de la section autour de l'axe x , tandis que le terme $u_x = \alpha \omega(y, z)$ décrit le déplacement axial additionnel nécessaire pour satisfaire les conditions de compatibilité et d'équilibre.

La fonction ω dépend uniquement de la géométrie de la section. Sa détermination constitue l'inconnue principale du problème de torsion.

2.2 Déformations et contraintes de cisaillement

Sous les hypothèses des petites perturbations, les seules composantes de déformation non nulles sont les cisaillements :

$$\gamma_{xy} = \frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x}, \quad \gamma_{xz} = \frac{\partial u_x}{\partial z} + \frac{\partial u_z}{\partial x}. \quad (2)$$

En substituant le champ de déplacement précédent :

$$\boxed{\gamma_{xy} = \alpha \left(\frac{\partial \omega}{\partial y} - z \right), \quad \gamma_{xz} = \alpha \left(\frac{\partial \omega}{\partial z} + y \right)} \quad (3)$$

Dans un matériau isotrope vérifiant la loi de Hooke :

$$\tau_{xy} = G \gamma_{xy}, \quad \tau_{xz} = G \gamma_{xz}, \quad (4)$$

où G est le module de cisaillement.

On obtient alors :

$$\boxed{\tau_{xy} = G \alpha \left(\frac{\partial \omega}{\partial y} - z \right), \quad \tau_{xz} = G \alpha \left(\frac{\partial \omega}{\partial z} + y \right)} \quad (5)$$

Ces relations montrent que les contraintes de cisaillement sont directement pilotées par les gradients de la fonction de gauchissement.

2.3 Équation forte

Dans le cas d'une torsion uniforme sans forces volumiques, l'équilibre mécanique conduit à :

$$\frac{\partial \tau_{xy}}{\partial y} + \frac{\partial \tau_{xz}}{\partial z} = 0. \quad (6)$$

En injectant les expressions (5) :

$$\frac{\partial}{\partial y} \left(\frac{\partial \omega}{\partial y} - z \right) + \frac{\partial}{\partial z} \left(\frac{\partial \omega}{\partial z} + y \right) = 0. \quad (7)$$

Comme :

$$\frac{\partial(-z)}{\partial y} = 0, \quad \frac{\partial y}{\partial z} = 0,$$

on obtient :

$$\boxed{\Delta \omega = \frac{\partial^2 \omega}{\partial y^2} + \frac{\partial^2 \omega}{\partial z^2} = 0 \quad \text{dans } \Omega} \quad (8)$$

Sur le contour libre $\partial\Omega$, le vecteur contrainte tangentielle doit être tangent au bord. Cette condition conduit à la condition naturelle :

$$\boxed{\frac{\partial \omega}{\partial n} = z n_y - y n_z \quad \text{sur } \partial\Omega} \quad (9)$$

où $\mathbf{n} = (n_y, n_z)$ désigne la normale extérieure.

Le problème est donc un problème de Neumann pur : la solution est définie à une constante additive près. Une condition supplémentaire doit être imposée pour garantir l'unicité.

2.4 Formulation variationnelle

Afin d'appliquer la méthode des éléments finis, le problème est reformulé sous forme faible.

Soit une fonction test $v \in H^1(\Omega)$. On multiplie l'équation forte par v puis on intègre sur le domaine :

$$\int_{\Omega} \Delta \omega v \, d\Omega = 0. \quad (10)$$

Une intégration par parties (formule de Green) donne :

$$-\int_{\Omega} \nabla \omega \cdot \nabla v \, d\Omega + \int_{\partial\Omega} \frac{\partial \omega}{\partial n} v \, d\Gamma = 0. \quad (11)$$

En utilisant la condition de Neumann :

$$\frac{\partial \omega}{\partial n} = z n_y - y n_z,$$

on obtient :

$$\int_{\Omega} \nabla \omega \cdot \nabla v \, d\Omega = \int_{\partial\Omega} (z n_y - y n_z) v \, d\Gamma. \quad (12)$$

En appliquant ensuite le théorème de Green au terme frontière, on obtient finalement :

$$\boxed{\int_{\Omega} \nabla \omega \cdot \nabla v \, d\Omega = \int_{\Omega} \left(-z \frac{\partial v}{\partial y} + y \frac{\partial v}{\partial z} \right) d\Omega \quad \forall v \in H^1(\Omega)} \quad (13)$$

Cette formulation variationnelle constitue la base de la discrétisation par éléments finis développée dans la suite du rapport.

3 Discrétisation par éléments finis

3.1 Approximation par éléments triangulaires CST

Le domaine Ω est discrétisé en N_{el} éléments triangulaires linéaires à trois nœuds (CST — *Constant Strain Triangle*).

Chaque élément e possède trois nœuds de coordonnées :

$$(y_i, z_i), \quad i = 1, 2, 3.$$

Sur un élément triangulaire linéaire, la fonction de gauchissement est approchée par une interpolation affine :

$$\boxed{\omega^e(y, z) = a + b y + c z} \quad (14)$$

Cette approximation peut également s'écrire sous forme nodale :

$$\boxed{\omega^e(y, z) = N_1(y, z) \omega_1 + N_2(y, z) \omega_2 + N_3(y, z) \omega_3} \quad (15)$$

où :

- ω_i sont les valeurs nodales de la fonction de gauchissement,
- $N_i(y, z)$ sont les fonctions de forme linéaires du triangle.

Les coefficients (a, b, c) sont reliés aux valeurs nodales par :

$$\begin{pmatrix} 1 & y_1 & z_1 \\ 1 & y_2 & z_2 \\ 1 & y_3 & z_3 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{pmatrix}. \quad (16)$$

En notant :

$$M = \begin{pmatrix} 1 & y_1 & z_1 \\ 1 & y_2 & z_2 \\ 1 & y_3 & z_3 \end{pmatrix}, \quad (17)$$

on obtient :

$$\begin{pmatrix} a \\ b \\ c \end{pmatrix} = M^{-1} \begin{pmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{pmatrix}. \quad (18)$$

3.2 Gradient discret et matrice B

L'un des avantages majeurs des éléments CST est que le gradient de la solution est constant sur chaque élément.

En effet :

$$\nabla \omega^e = \begin{pmatrix} \frac{\partial \omega^e}{\partial y} \\ \frac{\partial \omega^e}{\partial z} \end{pmatrix} = \begin{pmatrix} b \\ c \end{pmatrix}. \quad (19)$$

À partir de la relation précédente :

$$\boxed{\nabla \omega^e = B \boldsymbol{\omega}^e} \quad (20)$$

avec :

$$\boldsymbol{\omega}^e = \begin{pmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{pmatrix},$$

et :

$$\boxed{B = \underbrace{\begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}}_{\text{gradN}} M^{-1}} \quad (21)$$

La matrice B joue le rôle d'opérateur gradient discret. Comme les fonctions de forme sont linéaires, B est constant sur chaque élément.

3.3 Formulation élémentaire

La formulation variationnelle obtenue précédemment est :

$$\int_{\Omega} \nabla \omega \cdot \nabla v \, d\Omega = \int_{\Omega} \left(-z \frac{\partial v}{\partial y} + y \frac{\partial v}{\partial z} \right) d\Omega. \quad (22)$$

Après discrétisation EF, la solution et les fonctions test sont interpolées par les mêmes fonctions de forme.

Sur un élément e :

$$\nabla \omega^e = B \boldsymbol{\omega}^e, \quad \nabla v^e = B \mathbf{v}^e. \quad (23)$$

Le terme bilinéaire conduit à la matrice de rigidité élémentaire :

$$K^e = \int_{\Omega_e} B^T B \, d\Omega. \quad (24)$$

Comme B est constant sur l'élément CST :

$$K^e = S^e B^T B \quad (25)$$

où S^e désigne l'aire du triangle :

$$S^e = \frac{1}{2} |(y_2 - y_1)(z_3 - z_1) - (y_3 - y_1)(z_2 - z_1)| \quad (26)$$

3.4 Vecteur de charge élémentaire

Le second membre de la formulation faible est :

$$f^e = \int_{\Omega_e} \left(-z \frac{\partial v}{\partial y} + y \frac{\partial v}{\partial z} \right) d\Omega. \quad (27)$$

Comme le gradient des fonctions de forme est constant, une intégration numérique à un point de Gauss est exacte pour les éléments CST.

Le point d'intégration choisi est le centroïde :

$$(y_c, z_c) = \frac{1}{3} \sum_{i=1}^3 (y_i, z_i) \quad (28)$$

Le vecteur de charge élémentaire devient alors :

$$\mathbf{q}^e = S^e B^T \begin{pmatrix} -z_c \\ y_c \end{pmatrix} \quad (29)$$

3.5 Assemblage global

Les contributions élémentaires sont assemblées dans le système global :

$$K \boldsymbol{\omega} = \mathbf{f} \quad (30)$$

où :

- K est la matrice globale de rigidité,
- $\boldsymbol{\omega}$ contient les inconnues nodales,
- $\mathbf{f}$ est le vecteur second membre global.

L'assemblage consiste à additionner les contributions des éléments partageant les mêmes degrés de liberté nodaux.

3.6 Condition d'unicité

Le problème étudié est un problème de Neumann pur : la solution est définie à une constante additive près. La matrice globale K est donc singulière.

Pour lever cette indétermination, une condition supplémentaire est imposée :

$$\boxed{\omega(i_c) = 0} \quad (31)$$

où i_c désigne le nœud le plus proche du centroïde géométrique du maillage.

Numériquement, cela revient à remplacer la ligne correspondante du système par :

$$K_{i_c, \cdot} = \mathbf{e}_{i_c}^T, \quad f_{i_c} = 0. \quad (32)$$

Ce choix permet :

- d'assurer l'unicité de la solution,
- d'éviter les oscillations numériques,
- de préserver la symétrie physique du problème.

4 Implémentation Fortran

4.1 Structure du programme

Le solveur est organisé en six étapes :

1. Lecture du maillage via le module `mesh.f90`
2. Allocation de $K \in \mathbb{R}^{N_{no} \times N_{no}}$ et $\mathbf{f} \in \mathbb{R}^{N_{no}}$
3. Boucle d'assemblage sur les N_{el} éléments
4. Imposition de la condition $\omega(i_c) = 0$
5. Résolution par factorisation LU (LAPACK : `dgetrf/dgetrs`)
6. Export au format Gmsh `resu.msh`

4.2 Boucle d'assemblage

Listing 1: Extrait de la boucle d'assemblage — `TP_elas.f90`

```

1  do i = 1, nelelem
2      ! Coordonnees recentrees des noeuds de l'element
3      y1 = coor(1,n1) ; z1 = coor(2,n1)
4      ...
5      ! Matrice de geometrie et son inverse
6      M_mat(1,:) = [1.0d0, y1, z1]
7      call invermat(3, M_mat, M_inv)
8      ! Matrice gradient : B = gradN * M^{-1}
9      B_mat = matmul(gradN, M_inv)
10     ! Contributions elementaires
11     Kel = selem * matmul(transpose(B_mat), B_mat)
12     qel = selem * matmul(transpose(B_mat), [-z_c, y_c])
13     ! Assemblage dans K et f globaux
14     Rsys(nodelem(r,i), nodelem(c,i)) += Kel(r,c)
15     vecs(nodelem(r,i), 1) += qel(r)

```



```
16 end do
```

5 Résultats et interprétation

5.1 Maillage

La section rectangulaire étudiée est de dimensions $y \in [-0.05, 0.05]$ m (direction courte) et $z \in [-0.4375, 0.4375]$ m (direction longue), soit un rapport d'élancement $L/e \approx 9$. Le maillage par défaut comporte $N_{no} = 316$ nœuds et $N_{el} = 532$ éléments triangulaires CST.

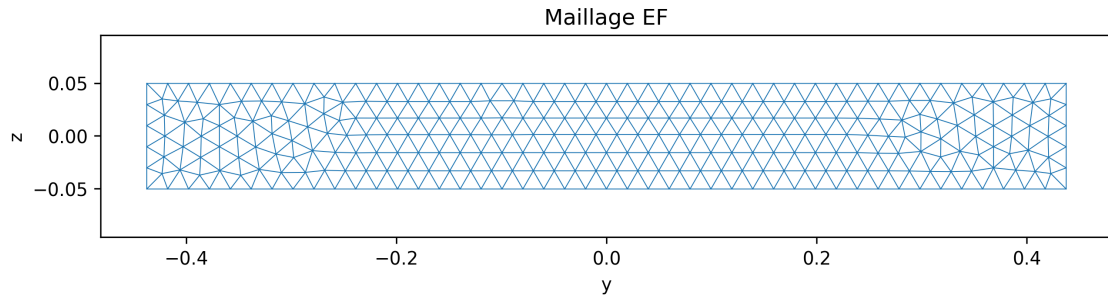


Figure 1: Discrétisation de la section rectangulaire par éléments triangulaires CST.

5.2 Fonction de gauchissement ω

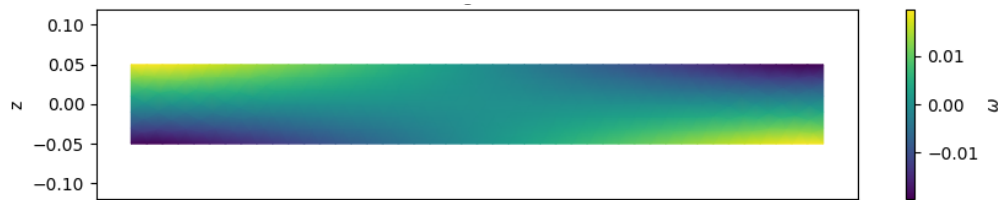


Figure 2: Distribution 2D de la fonction de gauchissement $\omega(y, z)$.

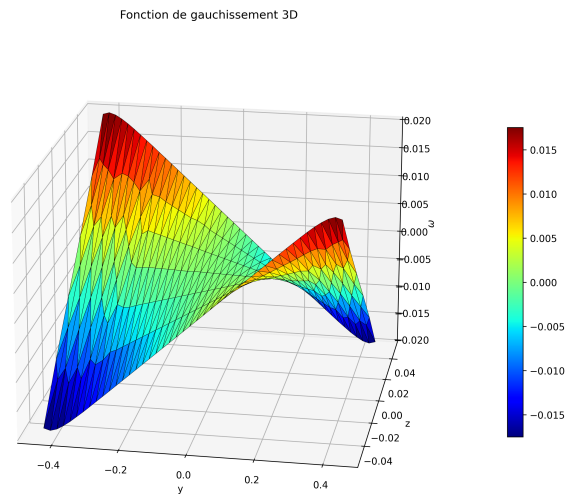


Figure 3: Vue 3D de $\omega(y, z)$. La forme en selle de cheval est caractéristique du gauchissement d'une section rectangulaire.

La solution numérique présente les propriétés de symétrie attendues :

- **Antisymétrie en y** : $\omega(-y, z) = -\omega(y, z)$,
- **Antisymétrie en z** : $\omega(y, -z) = -\omega(y, z)$,
- **Extrema aux coins** : $|\omega|_{\max} \approx 0.0196 \text{ m}^2$.

Pour une section très élancée ($L/e \gg 1$), la solution analytique approche $\omega \approx -yz$, ce qui prédit des extrema aux coins valant $|y_{\max} \times z_{\max}| = 0.05 \times 0.4375 = 0.02188 \text{ m}^2$. La valeur FEM de 0.0196 m^2 est légèrement inférieure, ce qui s'explique par la correction apportée par les termes de la série de Saint-Venant qui atténuent le gauchissement près des bords courts.

5.3 Profils de ω

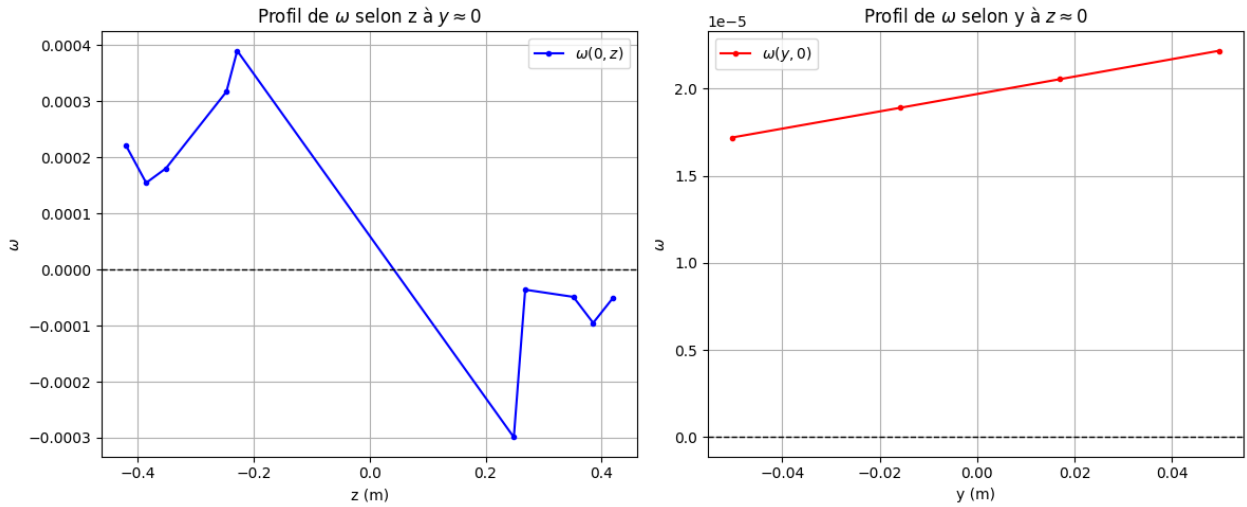


Figure 4: Profils 1D de la fonction de gauchissement. (gauche) $\omega(0, z)$: profil selon z à $y \approx 0$, quasi-nul conformément à l'antisymétrie de la solution ; (droite) $\omega(y, 0)$: profil selon y à $z \approx 0$, valeurs de l'ordre de 10^{-5} m^2 (essentiellement zéro).

Les deux profils confirment que $\omega(0, z) \approx 0$ et $\omega(y, 0) \approx 0$, ce qui est cohérent avec l'antisymétrie théorique de la fonction de gauchissement pour une section rectangulaire centrée à l'origine. Le bruit résiduel sur le profil $\omega(0, z)$ est dû à la discrétisation : les nœuds sélectionnés à $y \approx 0$ ne sont pas exactement sur l'axe de symétrie.

5.4 Contraintes de cisaillement

À partir de ω , les contraintes de cisaillement sont dérivées par :

$$\tau_{xy} = G \alpha \left(\frac{\partial \omega}{\partial y} - z \right), \quad \tau_{xz} = G \alpha \left(\frac{\partial \omega}{\partial z} + y \right) \quad (33)$$

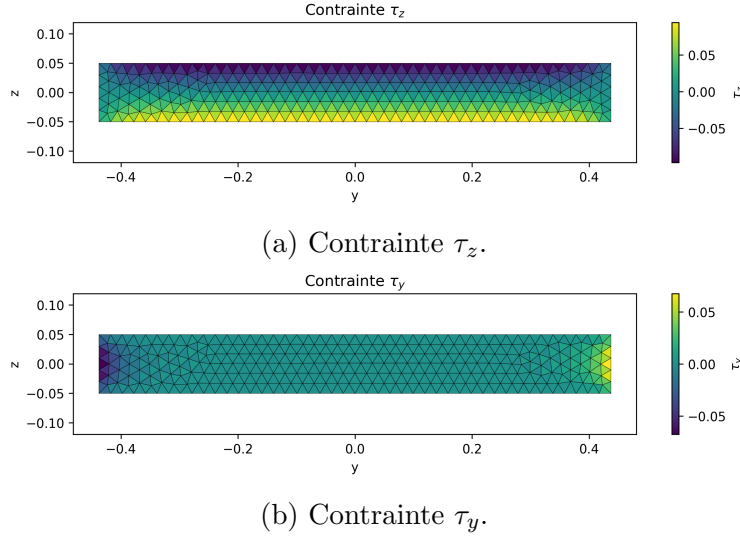


Figure 5: Distribution des contraintes de cisaillement sur la section.

Interprétation physique. Contrainte τ_z — Elle présente une variation quasi-linéaire selon z (terme dominant $-z$ dans τ_{xy}), avec des valeurs maximales aux bords supérieur et inférieur ($z = \pm z_{\max}$). Ce comportement est cohérent avec la théorie élémentaire de la torsion pour une section élancée.

Contrainte τ_y — Elle est quasi-nulle sur la majeure partie de la section et concentrée aux extrémités $z = \pm z_{\max}$ (bords longs). Pour une section $L/e \approx 9$, la torsion est essentiellement résistée par τ_z ; la contribution de τ_y est un effet de bord localisé.

Ce résultat est physiquement attendu : dans une section très élancée, le comportement se rapproche de celui d’une plaque mince en torsion, où seule la contrainte perpendiculaire à la grande dimension est significative.

6 Conclusion

Ce travail a permis de construire et valider une chaîne de calcul éléments finis complète pour le problème de gauchissement de Saint-Venant. Les résultats obtenus sont en accord avec les prédictions théoriques :

- La fonction ω présente la forme en selle caractéristique et les antisymétries attendues,
- Les extrema aux coins ($\approx 0.0196 \text{ m}^2$) sont proches de la valeur limite $yz_{\max} = 0.02188 \text{ m}^2$, confirmant la cohérence physique de la solution,
- Les contraintes de cisaillement sont dominées par τ_z et cohérentes avec le fort élanement de la section.

Des améliorations envisageables incluent l’utilisation d’éléments $\mathcal{P}_2$ (6 nœuds) pour accélérer la convergence, ou l’application d’un lissage SPR (*Superconvergent Patch Recovery*) pour améliorer la régularité inter-éléments des contraintes.

A Code source — TP_elas.f90

Le code est présenté en cinq blocs distincts correspondant aux étapes du solveur.

A.1 — Déclarations et initialisation

Listing 2: Déclarations et lecture du maillage

```

1  program TP_elas
2      use mesh
3      implicit none
4
5      integer  :: i, r, c
6      integer  :: n1, n2, n3
7      integer  :: i_center
8      real(8)  :: ybar, zbar, dist, dist_min
9      real(8)  :: y1, y2, y3, z1, z2, z3, selem, y_c, z_c
10     real(8)  :: gradN(2,3), M_mat(3,3), M_inv(3,3)
11     real(8)  :: B_mat(2,3), Kel(3,3), qel(3)
12     real(8), allocatable :: Rsys(:,,:), vecs(:,,:), dep(:,,:), coor(:,,:)
13
14     nord = 1
15     call rw_mesh      ! lecture du fichier MAIL.msh
16
17     allocate(Rsys(nno,nno), vecs(nno,1), dep(nno,1), coor(2,nno))
18     Rsys = 0.0d0 ; vecs = 0.0d0 ; dep = 0.0d0
19
20     ! Operateur gradient de reference : extrait dN/dy et dN/dz
21     gradN      = 0.0d0
22     gradN(1,2) = 1.0d0      ! ligne 1 -> d/dy
23     gradN(2,3) = 1.0d0      ! ligne 2 -> d/dz

```

A.2 — Nœud central et coordonnées recentrées

Listing 3: Recherche du nœud central et recentrage

```

1      ! Centroide geometrique du maillage
2      ybar = sum(ym) / nno
3      zbar = sum(zm) / nno
4
5      ! Noeud le plus proche du centroide -> impose omega = 0
6      i_center = 1
7      dist_min = (ym(1)-ybar)**2 + (zm(1)-zbar)**2
8      do i = 2, nno
9          dist = (ym(i)-ybar)**2 + (zm(i)-zbar)**2
10         if (dist < dist_min) then
11             dist_min = dist
12             i_center = i
13         end if
14     end do
15
16     ! Coordonnees recentrees (pour l'export Gmsh)
17     coor(1,:) = ym - ybar
18     coor(2,:) = zm - zbar

```

A.3 — Boucle d'assemblage

Listing 4: Assemblage de la matrice de rigidité et du vecteur de charge

```

1  do i = 1, nelelem
2      n1 = nodelem(1,i) ; n2 = nodelem(2,i) ; n3 = nodelem(3,i)

```

```

3      y1 = coor(1,n1) ; z1 = coor(2,n1)
4      y2 = coor(1,n2) ; z2 = coor(2,n2)
5      y3 = coor(1,n3) ; z3 = coor(2,n3)
6
7      ! Matrice de geometrie M et son inverse
8      M_mat(1,:) = [1.0d0, y1, z1]
9      M_mat(2,:) = [1.0d0, y2, z2]
10     M_mat(3,:) = [1.0d0, y3, z3]
11     call invermat(3, M_mat, M_inv)
12
13     ! B = gradN * M^{-1} (gradient constant sur l'element)
14     B_mat = matmul(gradN, M_inv)
15
16     ! Aire de l'element
17     selem = 0.5d0 * abs((y2-y1)*(z3-z1) - (y3-y1)*(z2-z1))
18
19     ! Matrice de rigidite et vecteur de charge elementaires
20     Kel = selem * matmul(transpose(B_mat), B_mat)
21     y_c = (y1+y2+y3)/3.0d0 ; z_c = (z1+z2+z3)/3.0d0
22     qel = selem * matmul(transpose(B_mat), [-z_c, y_c])
23
24     ! Assemblage global
25     do r = 1, 3
26         do c = 1, 3
27             Rsys(nodelem(r,i), nodelem(c,i)) = &
28             Rsys(nodelem(r,i), nodelem(c,i)) + Kel(r,c)
29         end do
30         vecs(nodelem(r,i), 1) = vecs(nodelem(r,i), 1) + qel(r)
31     end do
32 end do

```

A.4 — Condition aux limites et résolution

Listing 5: Imposition de la CL et résolution du système linéaire

```

1      ! Condition aux limites : omega = 0 au noeud central
2      ! (leve l'indetermination du probleme de Neumann pur)
3      Rsys(i_center, :) = 0.0d0
4      Rsys(i_center, i_center) = 1.0d0
5      vecs(i_center, 1) = 0.0d0
6
7      ! Resolution : K * dep = vecs (factorisation LU via LAPACK)
8      call linsys(nno, 1, Rsys, vecs, dep)
9
10     ! Export au format Gmsh et affichage
11     call figgmshno_0(nno, nelem, nodelem, coor, dep(:,1))
12     write(*,*) 'Done.  nno=', nno, '  nelem=', nelem
13     write(*,*) 'omega_min=', minval(dep), '  max=', maxval(dep)

```

A.5 — Sous-routines mathématiques

Listing 6: Inversion de matrice et solveur linéaire (LAPACK)

```

1      contains
2
3      ! Inversion de matrice n x n via dgetrf + dgetri
4      subroutine invermat(n, raid, raidi)

```

```

5 integer, intent(in) :: n
6 real(8), intent(in) :: raid(n,n)
7 real(8), intent(out) :: raidi(n,n)
8 integer :: ipiv(n), info, nsize
9 real(8) :: workr(n)
10 real(8), allocatable :: work(:)
11 real(8) :: raidt(n,n)
12 raidt = raid
13 call dgetrf(n, n, raidt, n, ipiv, info)
14 call dgetri(n, raidt, n, ipiv, workr, -1, info)
15 nsize = int(workr(1)) ; allocate(work(nsize))
16 call dgetri(n, raidt, n, ipiv, work, nsize, info)
17 if (info /= 0) write(*,*) 'Warning:␣singular␣matrix'
18 raidi = raidt ; deallocate(work)
19 end subroutine
20
21 ! Solveur lineaire rai*dep = force via dgetrf + dgetrs
22 subroutine linsys(n, m, rai, force, dep)
23 integer*4 :: n, m
24 integer :: ipiv(2*n), info
25 real*8 :: rai(n,n), rail(n,n), dep(n,m), force(n,m)
26 rail = rai ; dep = force
27 call dgetrf(n, n, rail, n, ipiv, info)
28 call dgetrs('N', n, m, rail, n, ipiv, dep, n, info)
29 if (info /= 0) write(*,*) 'Warning:␣system␣not␣solved'
30 end subroutine

```

A.6 — Export Gmsh

Listing 7: Écriture du fichier resu.msh au format Gmsh 2.2

```

1      subroutine figmshno_0(nno, nelelem, nodelem, coor, dep)
2          integer, intent(in) :: nno, nelelem
3          integer, intent(in) :: nodelem(3,nelelem)
4          real(8), intent(in) :: coor(2,nno), dep(nno)
5          integer :: i
6          open(unit=1, file='resu.msh', form='formatted', status='unknown'
7              )
8          ! En-tete et noeuds
9          write(1,'(a)') '$MeshFormat' ; write(1,'(a)') '2.2_0_8'
10         write(1,'(a)') '$EndMeshFormat' ; write(1,'(a)') '$Nodes'
11         write(1,'(I6)') nno
12         do i = 1, nno
13             write(1,'(I6,3(1X,F20.10))') i, 0.0d0, coor(1,i), coor(2,i)
14         end do
15         write(1,'(a)') '$EndNodes'
16         ! Elements
17         write(1,'(a)') '$Elements' ; write(1,'(I6)') nelelem
18         do i = 1, nelelem
19             write(1,'(I6,"_2_2_1_1",3I6)') &
20                 i, nodelem(1,i), nodelem(2,i), nodelem(3,i)
21         end do
22         write(1,'(a)') '$EndElements'
23         ! Donnees nodales (omega)
24         write(1,'(a)') '$NodeData' ; write(1,'(a)') '1'
25         write(1,'(a)') '"omega"' ; write(1,'(a)') '1'
26         write(1,'(a)') '_0.0' ; write(1,'(a)') '3'

```

```
26      write(1,'(a)') '0'           ; write(1,'(a)') '1'
27      write(1,'(I6)') nno
28      do i = 1, nno
29          write(1,'(I6,1X,F20.10)') i, dep(i)
30      end do
31      write(1,'(a)') '$EndNodeData'
32      close(1)
33  end subroutine
34
35 end program TP_elas
```